Formal proofs as guardrails for LLM coding agents

Invariants as the durable artifact of a regular web SaaS application — agents write the code, reasoning engines hold it to the rules

Research review — workflow, evidence from 7 prototypes, 13 tracks, and two worked services
August 2026

No formal-methods background assumed — the four concepts you need are on slide 3.

WHY THIS PROJECT

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. Concurrency bugs live precisely in the interleavings you did not think of.
- If we want agents to optimize systems autonomously (performance, cost), we need a way to say what must never break — and have a machine enforce it.

The proposal

The developer writes down intent as invariants. Agents write and rewrite the code. Reasoning engines sit between them and arbitrate: every change must provably keep the invariants.

Code becomes cheap to regenerate. The spec becomes the durable asset.

Setting: a regular web SaaS application — CRUD/API handlers, authorization, tenancy, schema migrations running under live traffic. Not kernels, crypto, or avionics: every tool below is judged against ordinary product teams, CI budgets, and mainstream languages at the edges.

BACKGROUND IN FOUR WORDS

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “no request ever reads a database column that has been dropped.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves (a request commits, a migration step runs). Small enough to explore, faithful enough that its bugs are your bugs.

Checker

A tireless adversary. It plays every possible ordering of moves against the model — millions of interleavings you would never think to test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact move list that breaks the invariant. Not “something is wrong” but “this sequence of 9 steps ends with a stale read.” Perfect food for an LLM to repair against.

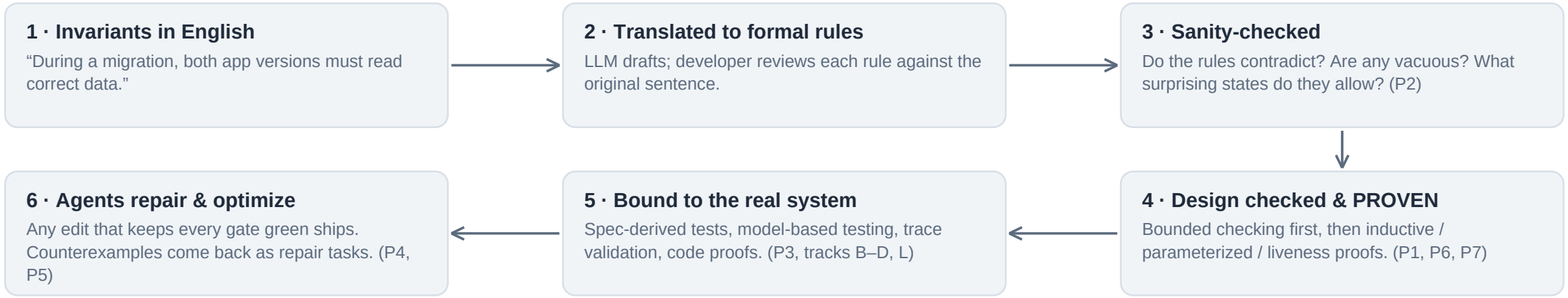
THE WORKFLOW

Three roles, one contract — every stage now built and exercised

DEVELOPER
owns intent

LLM AGENTS
own implementation

REASONING ENGINES
arbitrate



The contract: the spec is frozen for the agent — enforced mechanically (the harness diffs the frozen region, reverts, and fails the round), never by convention or prompt. Objectives (speed, cost) are optimized only inside the region the spec allows. The agent may make the system faster any way it likes; it may not make it wrong.

CASE STUDY

Changing the database schema while serving traffic

Why this problem: it is ubiquitous at scale, genuinely concurrent, and formally treated exactly once in the literature (Google F1, 2013). The popular open-source migration tools ship no correctness argument at all.

- Meanwhile: two versions of the application run side by side (rolling deploy). Version 1 has never heard of the new column.
- Meanwhile: user requests read and write the same rows the migration is copying — under snapshot isolation, so everyone sees a consistent-looking snapshot and conflicts surface only at commit.
- One protocol mistake in the choreography and some user reads return stale data — silently.

EVIDENCE · DESIGN LEVEL (P1)

The checker beat us — twice

We wrote the migration protocol as a small model (Quint) with three one-line invariants, believing each version correct. Random simulation found counterexamples in under a second; symbolic checking confirmed them.

Bug #1 — the drain guard was in the wrong place

Rule as written: “backfill may not start until every app instance runs v2.” Counterexample: dual-writes alone fill the new column, so the read switch fires with no backfill at all — while a v1 instance still lives. It writes the old column afterwards; a v2 user reads stale data.

Fix: the drain must gate the read switch too, not just the backfill.

Bug #2 — the textbook backfill query is wrong

Every guide says: backfill WHERE new_column IS NULL. Counterexample: during the rolling window a v2 instance dual-writes a row (new column now non-NULL), then a surviving v1 instance overwrites the old column only. The row is non-NULL but stale — and IS NULL never revisits it. The staleness survives to the switch.

Fix: backfill WHERE new IS DISTINCT FROM old; switch when all rows agree.

Both are real production failure modes of app-level dual-write migrations — and neither is in the literature or the OSS tools' docs. After the fixes: 30,000 random traces clean, Apalache verifies to 12 steps — and the protocol is now PROVEN at every depth and every system size (slide 12).

This is the workflow working as designed: the human states three one-line invariants; the machine finds the non-obvious consequences of the protocol.

EVIDENCE · RULE LEVEL (P2)

Asking the logic engine: do my rules even make sense?

Before any model or code exists, the invariant set itself can be interrogated (P2: a small CLI over the Z3 solver; rules are typed and plain enough to review sentence-by-sentence).

“Do any rules contradict?” IMPOSSIBLE

Two developers add “at most one app version at a time” and “at least two during rolling deploys.” The solver returns the minimal conflicting pair — by name — out of the whole rule set.

“What states do the rules allow?” SATISFIABLE

Enumerated example states reveal a forgotten guard: “dropping the column while an old binary still runs” was allowed. Nobody wrote a wrong rule — one rule was missing. The witness shows it.

“Does my claim follow from the rules?” VALID / INVALID

“Contract never starts while v1 runs” → VALID: the rules entail it, no test suite needed. The same query gates agent edits in P4/P5 today.

Each verdict is also emitted as JSON — the machine-readable half of the agent loop, now closed (slide 10).

EVIDENCE · REAL SYSTEM (P3)

The same invariant, enforced against real Postgres

P3: a real Rust API (two versions running side by side), a real Postgres database, the real five-step migration — exercised by generated concurrent traffic while the migration runs (Hypothesis test harness).

735
concurrent requests across all migration phases

0
errors with the proven choreography

59
“column does not exist” errors per run when the drain step is deliberately skipped

1
genuine race found in the ‘modern’ instance itself (and fixed)

- The deliberately-broken run reproduces exactly the anomaly the model predicted — the formal invariant is load-bearing, not decorative.
- Bonus find: a check-then-act race — the app reads migration flags, then executes SQL a moment later; a contract commit in that gap 500s a fully-migrated instance. The class of bug that survives code review and unit tests.
- Design-level proof (P1) and real-system evidence (P3) tell the same story about the same invariants.

WORKED EXAMPLE · A CMS

Ten security rules, one knob, every artifact runnable

The same workflow on a familiar domain (examples/cms): roles, draft articles, a review/publish lifecycle, public access. Stakeholder sentences become named, typed rules; the names travel from tickets to solver verdicts to model invariants to the app's 403 bodies.

cms_live — re-check auth on every action

20,000 simulated traces clean; Apalache verifies to depth 10; the safety invariant is INDUCTIVE with almost no strengthening — check-at-use is structurally the right design, and now that's a theorem (Track I). Policy suite over real HTTP: 5/5.

cms_cached — trust the session snapshot

Counterexample in under a second: author logs in, admin deactivates the account, the still-open session publishes anyway (stale JWT / cached claims). The same invariant provably has NO inductive proof here (concrete CTI). Replayed on the real app: 2/2 stale-token violations reproduced over HTTP.

- The rule oracle caught cross-ticket conflicts before any code: SEC-482 vs CMS-1201 (session length) flagged IMPOSSIBLE with the two rule names in the unsat core; LEG-77 silently killing review-before-publish flagged INVALID.
- Gates need BOTH directions: the happy-path feature test passes in both configurations — features don't catch the race, safety doesn't catch a feature dying. Every gate = safety + feature runs/witnesses.

REFRAMING · RULE-BASED SYSTEMS

Ground-up service: the rule base IS the program

The other framing, built (examples/rule-driven-cms, note 13): instead of writing code and holding it to a spec from outside, the developer writes ONE artifact — a rule base — and a generic, domain-free engine executes it. No handlers exist; the model↔code boundary disappears for the ruled part of the system.

241 : 975

lines of YAML that ARE the CMS vs lines of generic Python engine+analyzer — reused unchanged by a second service (tickets).

3,600 / 3,600

situations where runtime evaluation and the Z3 compilation of every rule agree — checked exhaustively, not sampled.

6 findings

against the FROZEN gate for two plausible edits: a privacy deny that kills review, a dropped separation-of-duties deny.

- Z3 reviews the rule base itself, per change: dead rules (never alter a decision — one redundant guard was proven useless and deleted), ∀-safety with the granting rule named, ∃-possibility with the blocking deny named, lifecycle liveness, frozen feature runs with expected denials by name.
- Same 403 vocabulary end to end: ticket sentence → rule id → solver finding → {"denied_by": rule_id} over real HTTP.
- The synthesis (note 13): rules are the programming surface, the solver is the reviewer, and proof effort concentrates on the once-proven engine — pattern 1 of note 12 (Cedar's shape), widened from authz to a whole service. Time, relations, and computation still escalate up the ladder.

THE AGENT LOOP CLOSED (P4)

Same bug, same model, same prompt — the gate decides

P4 seeds the historical IS-NULL bug into the migration protocol and lets a headless LLM repair it against the frozen spec (diffed mechanically, reverted on tamper). Run twice, changing ONLY the gate:

Episode 1 — safety-only gate

Repair in 1 round, invariants untouched, 30k traces + Apalache green — and WRONG in a way the gate couldn't see: the fix preserves safety by sacrificing completion (a stale row now blocks the switch forever). The safest system does nothing: a safety-only gate happily accepts it.

Episode 2 — gate + frozen feature runs & witness

Only change: adversarial stale-row-recovery run + completion-reachability witness added to the gate. Same generic prompt, same model: FULL fix in one round (IS-DISTINCT backfill + switch guard), 30k traces + Apalache clean, completion witnessed in 84% of traces.

- The lesson we now enforce as a guardrail: invest in gate strength, not natural-language steering. Weaker/cheaper models are fine when the gate is strong.
- Each counterexample becomes a frozen gate item — the gate ratchets, it never regresses.

Boundary by construction (Track G): protected app operations require a Grant<Op> capability token only the verified kernel can mint — bypassing authorization is now a pinned COMPILE ERROR (E0639/E0308), not a lint or a convention.

AUTONOMOUS OPTIMIZATION (P5)

The agent makes it 3.9x faster — and cannot make it wrong

The end-goal demo: an agent maximizes benchmark throughput on the CMS app (seeded with a global Mutex and fingerprinting-under-lock). The frozen gate: boundary lint + policy suite + race suites + spec-frozen paths enforced by git. The agent optimizes freely inside the fence.

3.94x

accepted speedup on the benchmark, all gates green

2

correctness-breaking “optimizations” attempted and mechanically absorbed by the gate

1

marginal edit rejected by the improvement threshold

- One rejected attempt was identity caching — exactly the stale-auth bug class the model, the code proof, and the race suite all encode. Three independent layers said no.
- Counterexamples, not humans, did the reviewing. The developer's only artifact is the spec.

ESCALATION: FROM TESTS TO THEOREMS

“Never happens” now means proven — at five levels

Bounded checking says “safe up to k steps at this size.” Phase 3 removed the qualifiers (all PROVEN, tracks I–M):

Inductive invariants — safe at EVERY depth

Track I · Quint/Apalache

7-conjunct strengthening for the migration protocol, ~10s to verify; CMS live-mode invariant inductive nearly for free, cached-mode provably has no such proof (concrete CTI). Strengthening effort is itself a design signal.

Parameterized — ANY number of keys & instances

Track J · mypyvy/EPR

Hand invariant verifies in 0.38s; UPDR independently INFERRED a 9-clause proof in 5.7s (machine-found, zero human strengthening). Negative control: the buggy variant provably has NO universal inductive invariant.

Liveness under fairness — it always completes

Track M · TLC + WF

<>(phase=DONE) proven over the complete 623-state graph, even under unbounded write interference — stronger than predicted. Dropping rollout fairness yields the stalled-deploy lasso: the assumption is necessary, and named.

Hyperproperties — no information leak

Track K · self-composition

“Draft contents never influence anonymous observations” relates PAIRS of traces — per-request checks cannot even state it. Proven inductively on the self-composed CMS; the leaky variant yields a machine-found two-world distinguisher.

Real code — the app's session/identity state machine

Track L · Dafny

Freshness, revocation & demotion immediacy proven (14/14 VCs) on logic extracted from main.rs; the cached-stale behavior is a machine-checked witness; a buggy resolver is rejected. Open gap: extraction fidelity (documented).

THE HONEST QUESTION

What binds the model to the code? A ladder, now fully exercised

Model↔code drift is the classic failure mode of industrial formal methods. Every rung below is implemented and measured in this repo (tracks B–E, G, L):

5 · Proofs in / code from the spec — proof

Dafny authorization kernel proven against all 10 rules, compiled to Go, embedded and demoed (Track D); the app's session logic proven (Track L); Grant<Op> makes bypass a compile error (Track G). Kani vs exhaustive measured (Track E): exhaustive wins at 64 points; Kani proves a 2^131-point domain in ~0.1s — adopt when ids/strings enter.

4 · Trace validation — strong evidence

Real app action logs compile into a generated Quint run: “was this a legal behavior of the model?” The cached-mode stale-token publish is accepted by the app but refused by the model — conformance violation caught from client-side logs alone (Track B).

3 · Model-based testing — strong evidence

Model-generated traces drive the real API: 240/240 steps at parity across 3 runs; all 6 replayed race counterexamples rejected by the live app at exactly the predicted step (Track C).

2 · Spec-derived tests against the real system — evidence

Migration harness 735 requests / 0 errors; CMS policy suite 5/5; deliberately-broken runs reproduce the model's predicted anomalies (P3).

1 · Shared vocabulary — traceability

One set of invariant names from tickets to rules to model to the app's 403 bodies. Proves nothing alone — makes every failure traceable to a requirement.

LANGUAGE SCOUTING

Is F* the endgame rule language? Verdict: no — but steal two ideas

F* is the purist endpoint of “rules as abstraction”: the type IS the rule (dependent/refinement types, SMT-checked, Pulse separation logic for concurrency; production-proven in Windows/Linux/Firefox crypto). Research note 11 evaluates it for our SaaS setting:

Why not primary

- LLM proof automation: ~1/3–1/2 of proofs (Microsoft's own FStarDataSet) vs Dafny's 86%; absent from the 2026 vericoding benchmark entirely.
- Extracts to OCaml/C/Rust — no web/SQL/framework story; could only ever be the kernel language, where Dafny already wins.
- Failed obligations are Z3 timeouts, not counterexamples — poison for a frozen gate where red must mean “your edit is wrong”.

Worth stealing

- The 3DGen pattern (Microsoft): agents write in a small constrained DSL; a pre-verified toolchain compiles it to provably correct code — ZERO prover calls per feature. Our translate/validate split, shipped.
- Effects & indexed types as by-construction boundaries — strictly stronger than our Grant<Op> tokens, if we ever need a richer kernel.

Falsifiable predictions logged (F1–F3), incl.: a DSL-with-verified-checker loop for tenant isolation beats the P4 gate on round count with zero per-feature proofs — testable today in our existing stack (P2's Z3 core). Since logged: industry confirmed the shape three times over — next slide.

SCALING RUNG 5

Five ways proofs meet code — ranked for an LLM-built SaaS (note 12)

1 · Verified engine + small DSL

zero proofs/feature

Prove the engine once, agents write rules in an analyzable DSL. AWS shipped this shape three times (Cedar with Lean-proven authorizer + sound-and-complete symbolic analysis, Bedrock AR checks, AgentCore's NL→Cedar loop) — and we now have it end-to-end: examples/rule-driven-cms runs a whole CMS from one rule base on a generic engine, Z3 gating every rule change (note 13).

2 · Proven kernel, generated & embedded

cloud-scale proof

Track D at scale: AWS rewrote IAM authorization in Dafny→Java — ~10^9 authorizations/second, spec validated by shadow-testing 10^15 production samples. Dafny is the LLM sweet spot: 86–93% on proof benchmarks; its top toolchains independently converged on our frozen-spec diff-check (guardrail 1).

3 · Proofs in the code — Rust only, honestly

now CI-priced

VeruSAGE: agents complete 81% of 849 proof tasks from 8 real verified systems at ~$5.61 and ~7 min per task. Flux adds a cheap refinement-type tier (specs on signatures, inference does the rest). Go costs 4–10 spec lines per code line; TypeScript and Python have no sound static story — the honest answer is Rust or nothing.

4 · Types as by-construction boundaries

zero CI cost

Track G's Grant<Op> tokens: ordinary compiler, no solver. Composes under every other pattern — the kernel's guarantees only reach the app if bypass is a compile error.

5 · Verified runtime enforcement

two gaps to own

Monitor synthesis is production-thin. Found instead: NO invariant→Postgres-constraint compiler exists (theory ready: invariant confluence), and NO verified schema-migration tool exists — our P1 + tracks I/J/M is the state of the art. New falsifiers R1–R5, prototypes P8 (Cedar shootout) and P9 (constraint compiler).

Nobody has published LLM-generated, machine-verified authz/billing/tenancy/migration logic — every production verified artifact above was hand-written by experts. That unoccupied intersection is this project's lane.

GUARDRAILS · CLAUDE.MD

What we learned the hard way — now enforced

1 · The spec is frozen for agents

Enforced mechanically — diff the frozen region, revert, fail the round. Never by convention or prompt.

2 · Gates need both directions

Safety-only gates accept fixes that trade away liveness (“the safest system does nothing”). Every gate = safety + feature runs + proof obligations.

3 · Gate strength beats NL steering

Same bug, model, prompt: the stronger gate turned a partial fix into the full fix. Cheaper models are fine when the gate is strong.

4 · Escalate to proofs

Bounded < inductive (any depth) < parameterized (any size) < liveness under fairness < hyperproperties via self-composition.

5 · Translate / validate split

LLM translation of NL→formal is unsound — always followed by a sound solver step. Humans review specs, never agent edits.

6 · Counterexamples are the currency

Machine-readable (ITF JSON, unsat cores, named 403s); one invariant vocabulary from ticket to runtime error.

7 · Verify sub-agent claims

The checker corrected the author several times — that is the method working. Dead ends and falsified predictions are recorded.

8 · Boundaries by construction

Capability tokens only the verified kernel can mint — a compile error, not a lint. Typestate over discipline.

ROADMAP

What's proven, what's parked, what's next

Done — 13 tracks, every falsifier tested

- Repair loop (A/F), trace validation (B), model-based testing (C), Dafny→Go kernel (D), Kani spike (E), compile-error boundary (G), gated optimization 3.94x (H).
- Proofs: inductive (I), parameterized w/ machine-inferred invariant (J), noninterference (K), real-code session proof (L), liveness under fairness (M). F* scouting (note 11).
- Rule-driven CMS (note 13): a whole service as one rule base on a generic engine; Z3 gates every rule change; frozen gate rejects two planted edits with 6 named findings.

Next

- P8 — Cedar shootout (note 12, R1): the same CMS rules as Cedar policies vs our native rule engine — plus note 13's falsifiers: LLM ticket→rule fidelity (RB3), the temporal twin that should catch the demoted-author race the rules can't see (RB4).
- P9 — invariant→Postgres compiler (R2): constraints/triggers where sound, isolation side-conditions explicit, residual obligations routed to the model checker. No such tool exists anywhere.
- Track N — refinement mappings (serializable refines SI; app-shaped refines abstract) · Track O — verified runtime enforcement · F3 DSL loop, now industry-backed · Flux spike (R5).
- TCB accounting as a first-class artifact — every “proven” names what it trusts (note 12 has the per-pattern table) and directs the next escalation.

Parked (consciously, with reasons recorded)

Alloy (P2/Z3 covers it) · Quint→Rust codegen (no tooling exists) · Kani for finite kernels (exhaustive is simpler; adopt when unbounded ids/strings arrive) · Verus (release downloads proxy-blocked; Dafny stands in).

The one-sentence takeaway

Write intent once, formally, with machine help —
then let agents rewrite the code forever,
with a tireless adversary guaranteeing they never make it wrong.

Repo: research/INDEX.md (all notes) · research/09, 10 & 12 (scoreboards with falsifiers) · CLAUDE.md (guardrails)
Prototypes: p1 check.sh · p2 demo.sh · p3 run_demo.sh · p4 agent loop · p5 optimization loop · p6 mypyvy · p7 TLC
Worked examples: examples/cms (spec beside code) · examples/rule-driven-cms (the rules ARE the code); all runnable